API DOCUMENTATION

MTI Radio Management API v1

A JSON REST API for other applications: the radio register, history, master data, users, audit log, SharePoint documents, E-Sign, and a health check for monitors.

Application version: 1.21.4 · **Date:** 21 October 2026

For: developers of applications that connect to MTI Radio Management (for example Power BI reports or other internal applications).

The sample answers in this document come from a copy of the application with made-up data, not from real radio data.

Contents

1. Overview 3
2. Authentication and tokens 4
3. Permissions 5
4. Conventions, errors and request limits 6
5. Account, references and dashboard 9
6. Radio register 10
7. History and import 15
8. Master data 16
9. Users, roles and audit log 17
10. SharePoint documents 18
11. E-Sign 21
12. Health check 22
13. Example workflows 24

1. Overview

The MTI Radio Management API gives other applications access to the radio register and its supporting data over HTTPS, in JSON. Its rules and permissions are those of the web pages: what a page refuses, the API refuses too.

Base address	`https://<app-address>/api/v1`: the address of the application's web pages followed by `/api/v1`
Format	JSON (UTF-8). Send the header `Accept: application/json`; POST and PUT bodies are JSON, except the Excel import (multipart/form-data)
Authentication	A token in the header `Authorization: Bearer <token>` (chapter 2)
Version	While the application is at version 1.x, the API stays at `/api/v1` and only grows: existing endpoints and fields are neither removed nor given another meaning
Language	Messages are in English; send `Accept-Language: id` for Indonesian. Data stays as stored, in Indonesian (chapter 4)
Time zone	WITA (+08:00)
Interactive documentation	`https://<app-address>/docs/api`: generated from the code, with a "Try it" button, for accounts that may hold a token

Endpoint groups

Group	Endpoints	Menu (permission)
Account and references	`me`, `references`, `dashboard`	– / Dashboard
Radio register	`radios` with archive, restore, history, free IDs, check, export, checkout form	Register Radio
History and import	`history`, `import`	Radio History, Import Excel
Master data	`radio-types`, `companies`, `departments`, `channel-templates`	The menu of each master table
Users, roles, audit	`users`, `roles`, `audit-log` (read only)	Registered Users, Role & Permission, Audit Log
SharePoint documents	`documents/cof-scan`, `documents/ict-masterlist`, `documents/edit-requests`	COF Scan, ICT Masterlist, Edit Approvals
E-Sign	`e-signs`	E-Sign Master
Health check	`health`, and `/up` outside `/api/v1`	No menu

Example requests

```
# curl
curl -H "Authorization: Bearer <token>" -H "Accept: application/json" \
     "https://<app-address>/api/v1/radios?per_page=2"

# PowerShell
$headers = @{ Authorization = "Bearer <token>"; Accept = "application/json" }
Invoke-RestMethod -Uri "https://<app-address>/api/v1/me" -Headers $headers
```

2. Authentication and tokens

Every request carries a token in the header `Authorization: Bearer <token>`. The token acts as its account, with that account's role; a browser's sign-in session does not open the API.

Creating a token

1. Open **Settings** → **API Tokens** in the application (in Indonesian: Pengaturan → API Tokens).
2. Create the token: give it a name, choose **read only** or **read & write** access, and a validity of 30, 90, 180 or 365 days.
3. Copy the token shown. It is shown only once; the application keeps only its hash.

Kind	Acts as	Created by	Stops when
API client	The account of an application (for example "Power BI") with one role, never Super Admin. It cannot sign in on screen and does not show in User Management	Super Admin	It expires, is revoked, or the client is deactivated or deleted
Personal token	Its owner, with the owner's role	Its owner (needs permission on the API Tokens menu)	It expires, is revoked, or the owner's account is deactivated

- A token is valid until the end of its last day, and can be revoked at any time on the API Tokens page. That page also shows when, and from which IP address, a token was last used.
- Tokens start with `mtiradio_`, so that a stray one is easy to recognise. Send the whole token exactly as it was shown when it was created.
- Keep a token like a password: never in program code, a repository or a shared document.
- The owner of a token gets a notification in the application 7 days before the token expires.
- Creating, changing and revoking a token are recorded in the Audit Log (its name only, never the token).

Checking a token

GET `/me`

No menu: every token may call it

The account and token of this request: who is represented, what it may do, and until when it is valid. `kind` is `person` or `client`.

```
{
  "data": {
    "account": { "id": 26, "name": "Browser Check", "kind": "person",
                 "email": "browser.check@example.test", "roles": ["Super Admin"] },
    "token": { "id": 8, "name": "HTTP check", "abilities": ["read", "write"],
               "expires_at": "2026-11-18T23:59:59+08:00", "last_used_at": "2026-10-19T16:30:54+08:00" }
  }
}
```

> **CORS closed:** the web pages of other sites cannot call the API from their visitors' browsers, unless an admin lists that site in the `CORS_ALLOWED_ORIGINS` setting. Applications that call the API from their server are not affected.

3. Permissions

Every endpoint belongs to the menu of its page, and needs the permission that matches its method on that menu, in the role of the token's account. Permissions are set in **Role & Permission**, as for the pages.

Method	Permission needed	Note
GET	Read	Reading, including the Excel export and the checkout form
POST	Create	Except archiving, restoring and adding radio history: Update, like their buttons on the page
PUT	Update	–
DELETE	Delete	–

- A **read only** token is refused (403) on every POST, PUT and DELETE.
- An import with `fresh=1` (replacing the whole register) also needs Delete on Register Radio.
- Approving and rejecting changes to workbook cells takes a personal token, and nobody can approve their own request.
- Users, roles and the audit log can only be read through the API; they are changed on their pages.
- Unlike the pages, an endpoint whose menu is not in the database is always closed.

A 403 message names the menu that lacks the permission, with the menu's name in the language of the request, so that an admin knows what to tick in Role & Permission:

```
{ "message": "The role of this account may not change on \"Register Radio\" (Update)." }
{ "message": "The role of this account has no access to \"Registered Users\"." }

with Accept-Language: id:
{ "message": "Role akun ini tidak boleh mengubah di \"Register Radio\" (Update)." }
{ "message": "Role akun ini tidak punya akses ke \"User Terdaftar\"." }
```

4. Conventions, errors and request limits

Lists and pages

List endpoints send one page at a time: the parameters `page` (from 1) and `per_page` (1–100, default 25). The answer holds `data`, `links` and `meta`:

```
{
  "data": [ { "id": 6, "dmr_id": 201, ... }, { "id": 7, "dmr_id": 203, ... } ],
  "links": { "first": "https://<app-address>/api/v1/radios?page=1",
             "last": "https://<app-address>/api/v1/radios?page=9",
             "prev": null, "next": "https://<app-address>/api/v1/radios?page=2" },
  "meta": { "current_page": 1, "from": 1, "last_page": 9, "per_page": 2, "to": 2, "total": 17, ... }
}
```

Fetch the next page while `links.next` is not `null`.

Filters and sorting

- Filters with several values may be sent as a list (`type[]=1&type[]=2`) or separated by commas (`type=1,2`).
- `sort` chooses the column to sort by (the values each endpoint allows), `direction` is `asc` or `desc`.
- `search` looks for text; with several words, the result follows the same rule as on the page.

Dates and times

Send dates as `YYYY-MM-DD`. Answers use `YYYY-MM-DD` for dates and ISO 8601 with the time zone for times (`2026-10-18T23:52:42+08:00`). The cells of the ICT Masterlist workbook are sent as the page shows them (`dd-mm-yyyy`).

Language

The API's messages (`message`, `errors`, the reasons for a refusal, the sentences of the health check) are in English. A client that sends the header `Accept-Language: id` (also `id-ID`) gets Indonesian; any other language gets English.

Data is not translated: the labels of locations, conditions, marks of the Check column, archive reasons and kinds of history stay as stored in the register and in the Excel files, in Indonesian (`"label": "Di User"`), whatever the language of the request. The pages in English show them with English names (Di User is "With user" there). The codes of the values (`di_user`) never change.

Fixed values

The values allowed for fields and filters are in `GET /references` (chapter 5), with their labels as stored:

Field	Values
`placement` (Location)	`di_user` Di User · `stok_it` Stok IT · `service` Service · `hilang` Hilang
`condition` (Condition)	`layak` Layak · `perlu_service` Perlu Service · `rusak` Rusak · `belum_dicek` Belum dicek
`category` (radio type)	`HT` HT (Portable) · `RIG` Rig (Mobile)
`owner_group`	`mti` MTI · `non_mti` Non MTI
`archive_reason`	Pindah Site · Afkir · Hilang (BA selesai) · Lainnya
`action` (adding history)	Serah Terima · Service · Program · Verifikasi · Catatan

Field	Values
`issue` (Check column)	`needs_review` Perlu dicek · `sn_duplicate` SN dobel · `id_duplicate` Radio ID dobel · `id_hex_f` Radio ID mengandung F · `sn_empty` SN kosong · `sn_format` Format SN tidak standar · `id_empty` Radio ID kosong · `type_empty` Tipe radio kosong · `lost_no_ba` Hilang tanpa No. BA · `reprogram` Perlu program ulang · `verify_empty` Tgl verifikasi kosong · `verify_stale` Verifikasi lewat 12 bulan

Not overwriting someone else's change

Every radio carries a `version`. Send back the `version` of the last answer when you change a radio: if someone else changed it since you read it, nothing is saved and the answer is `409`. Read the radio again, then repeat the change on the latest data.

Status codes and errors

Code	Meaning
200, 201, 204	Success; 201 for data just created, 204 without content (for example after deleting)
401	No token, a wrong, expired or revoked token, or an inactive account
403	The role or the token may not do it (the message names the menu), or the token is read only
404	No such data
409	The radio was changed by someone else since it was read, the register is busy (for example an import is running), or the request has already been decided
422	Refused by a rule; `errors` holds the messages per field, as on the page, in the language of the request
429	Too many requests (see the limits below)
502, 504	SharePoint answered wrongly or too slowly (with a `reason`)
503	The API (or the Audit Log) is not switched on yet on the server, SharePoint cannot be reached (`reason: unavailable`), or the health check is `down` (chapter 12)

A 422, registering a radio with Radio ID 751 (MDC Code 2EF) at the location "With user" without a user name:

```
{
  "message": "Radio ID 751 cannot be used: its MDC Code (2EF) contains the letter F. Choose another ID. (and 1
more error)",
  "errors": {
    "dmr_id": [ "Radio ID 751 cannot be used: its MDC Code (2EF) contains the letter F. Choose another ID." ],
    "user_name": [ "User / unit name is required when the location is \"With user\"." ]
  }
}
```

The same request with `Accept-Language: id`:

```
{
  "message": "Radio ID 751 tidak bisa dipakai: MDC Code-nya (2EF) mengandung huruf F. Ganti dengan ID lain.
(and 1 more error)",
  "errors": {
    "dmr_id": [ "Radio ID 751 tidak bisa dipakai: MDC Code-nya (2EF) mengandung huruf F. Ganti dengan ID
lain." ],
    "user_name": [ "Nama User / Unit wajib diisi bila Posisi \"Di User\"." ]
  }
}
```

401 and 404, without and with `Accept-Language: id`:

```
401  { "message": "Unauthenticated." }      { "message": "Tidak terautentikasi." }
404  { "message": "Not found." }            { "message": "Tidak ditemukan." }
```

Request limits

What is limited	Limit
Every endpoint, per token	60 requests a minute
Excel import and export, per token	10 a minute
Endpoints that read SharePoint, per token	30 a minute
Approving cell changes, per token	10 a minute
Per IP address, before the token is checked	120 a minute
Health check `/api/v1/health`	Only the per-address limit above (counted together); `/up` is not limited

When a limit is exceeded, the answer is `429`; wait a moment and try again. To fetch the whole register, use `per_page=100` or a single `GET /radios/export`.

5. Account, references and dashboard

GET `/me`

No menu

The account and token of this request. The example is in chapter 2.

GET `/references`

No menu

The fixed values of the register with their labels as stored (Indonesian, whatever the language of the request): `placements`, `conditions`, `categories`, `owner_groups`, `archive_reasons`, `history_actions` (those that may be added by hand), `history_kinds` (every kind of history), `issues` (marks of the Check column) and `radio_id` (the limits of a Radio ID). Master data has its own endpoints (chapter 8).

```
{
  "data": {
    "placements": [ { "value": "di_user", "label": "Di User" }, { "value": "stok_it", "label": "Stok IT"
}, ... ],
    "archive_reasons": [ "Pindah Site", "Afkir", "Hilang (BA selesai)", "Lainnya" ],
    "history_actions": [ "Serah Terima", "Service", "Program", "Verifikasi", "Catatan" ],
    "radio_id": { "min": 1, "max": 16776415, "hex_f_refused": true },
    ...
  }
}
```

GET `/dashboard`

Menu Dashboard · Read

The dashboard figures for active radios: `total_active`, `total_archived`, the count per location (`placements`), per condition (`conditions`), per company (`by_company`), per type (`by_type`), the marks of the Check column (`issues`), and the next free Radio IDs (`next_free_dmr_ids`).

```
{
  "data": {
    "total_active": 17, "total_archived": 0,
    "placements": [ { "value": "di_user", "label": "Di User", "count": 17 }, ... ],
    "conditions": [ { "value": "layak", "label": "Layak", "count": 3 }, ... ],
    "by_company": [ { "company": "(kosong)", "count": 14 }, { "company": "MTI", "count": 3 } ],
    ...
  }
}
```

6. Radio register

A radio is one physical radio, known by its SN. Every endpoint of this chapter belongs to the **Register Radio** menu, and every change is recorded in the Radio History (in the name of the token's owner) and in the Audit Log.

A radio

Field	Content
`id`	The number of the radio in the database (used in the addresses of the endpoints)
`dmr_id`	Radio ID; `null` while the radio is archived (its ID is released)
`previous_dmr_id`	The Radio ID before it was archived, for when it is restored
`mdc_code`	The Radio ID in hexadecimal (1001 → "3E9")
`serial_number`	SN
`radio_type`, `owner_company`, `company`, `department`, `channel_template`	A master data object (`id`, name and other content) or `null`; `owner_company` is the asset owner, `company` the user's company
`owner_group`	"MTI" or "Non MTI", following the asset owner; `null` when it is not filled in
`user_name`	The name of the user or unit that holds it
`placement`, `condition`	An object `{ "value", "label" }`: location and condition
`last_programmed_at`, `last_verified_at`	The dates of the last programming and the last verification
`document_no`, `notes`	Document no. (PO / BA) and notes
`needs_review`, `review_notes`	The "Needs checking" mark from the Excel import, and its reason
`archived`, `archived_at`, `archive_reason`	Archive status
`issues`	The marks of the Check column for this radio (the values of `issue` in chapter 4)
`version`	Send it back when changing the radio (chapter 4)
`created_at`, `updated_at`	When it was created and last changed

GET `/radios`

Read · the list of radios, with the same filters as the Register Radio page

Parameter	Content
`search`	SN, user/unit, document no., notes, Radio ID or MDC Code (max. 100 characters)
`scope`	`active` (default), `archived` or `all`
`company[]`, `owner[]`, `department[]`, `type[]`, `template[]`	Ids of the user's company, asset owner, department, radio type, channel template (several allowed)

Parameter	Content
`owner_group`	`mti` or `non_mti`
`placement[]`, `condition[]`	Location and condition (several allowed)
`issue`	One mark of the Check column
`sort`, `direction`	`dmr_id`, `serial_number`, `user_name`, `last_programmed_at`, `last_verified_at`, `updated_at`; `asc` or `desc`
`page`, `per_page`	The page; 1–100 per page, default 25

GET `/radios/{radio}`

Read · one radio, archived ones included


```
{
  "data": {
    "id": 6, "dmr_id": 201, "previous_dmr_id": null, "mdc_code": "C9", "serial_number": "867PRV0201",
    "radio_type": { "id": 3, "name": "MotoTrbo XiR C2660", "category": "HT", "power": "5 W" },
    "owner_company": null, "owner_group": null, "user_name": "Andi Saputra",
    "department": null, "company": null, "channel_template": null,
    "placement": { "value": "di_user", "label": "Di User" },
    "condition": { "value": "belum_dicek", "label": "Belum dicek" },
    "last_programmed_at": null, "last_verified_at": "2026-10-18",
    "document_no": null, "notes": null, "needs_review": false, "review_notes": null,
    "archived": false, "archived_at": null, "archive_reason": null, "issues": [],
    "version": "20261018235242190",
    "created_at": "2026-10-18T23:51:38+08:00", "updated_at": "2026-10-18T23:52:42+08:00"
  }
}
```


POST `/radios`

Create · registers a radio; the 201 answer holds the new radio

Field	Required	Content and rules
`serial_number`	Yes	Text, max. 50, unique
`radio_type_id`	Yes	The id of a radio type (chapter 8)
`placement`	Yes	Location
`condition`	Yes	Condition
`dmr_id`	–	Radio ID 1–16776415: not used by an active radio, not reserved by an archived one, and with an MDC Code without the letter F
`user_name`	With user	Text, max. 150; required when `placement` = `di_user`
`owner_company_id`, `company_id`, `department_id`, `channel_template_id`	–	Ids of active master data
`last_programmed_at`, `last_verified_at`	–	Date `YYYY-MM-DD`

Field	Required	Content and rules
`document_no`	–	Text, max. 100
`notes`	–	Text, max. 5000

```
POST /api/v1/radios
{ "serial_number": "867DOC0001", "radio_type_id": 1, "placement": "stok_it", "condition": "layak",
  "notes": "Example from the API documentation" }
```

PUT `/radios/{radio}`

Update · changes a radio; the answer holds the latest radio

Send the fields that change (the same fields as POST); `placement` and `condition` are always sent, as in the form. Include the `version` of the last read: if someone else changed the radio, the answer is 409 and nothing is saved. `review_done: true` clears the "Needs checking" mark.

DELETE `/radios/{radio}`

Delete · a 204 answer without content

Only for a radio entered by mistake; its history stays. A radio that leaves for good is archived, not deleted.

POST `/radios/{radio}/archive`

Update · archives a radio; its Radio ID is released

Field	Required	Content
`archive_reason`	Yes	Pindah Site, Afkir, Hilang (BA selesai) or Lainnya (as stored)
`archived_at`	Yes	A date, today at the latest
`note`	–	A note, max. 1000

POST `/radios/{radio}/restore`

Update · restores an archived radio

Its old Radio ID comes back, unless another radio uses it now (then `dmr_id` stays empty).

GET /radios/{radio}/history

Read · the history of one radio, newest first


```
{
  "data": [ {
    "id": 4, "radio": { "id": 6, "display_name": "ID 201 · 867PRV0201", "user_name": "Andi Saputra",
"deleted": false },
    "event_date": "2026-10-18", "action": "Verifikasi", "description": "Stock opname gudang IT",
    "document_no": null, "changes": null, "user": null, "created_at": "2026-10-18T23:52:42+08:00"
  } ]
}
```


`changes` holds the changes of data per field (`label` , `old` , `new`); `user` is `null` for history written by the system (the Excel import).

POST /radios/{radio}/history

Update · adds one line of history; a 201 answer

Field	Required	Content
`action`	Yes	Serah Terima, Service, Program, Verifikasi or Catatan (as stored)
`event_date`	Yes	A date, today at the latest
`description`	Yes	A description, max. 2000
`document_no`	–	Max. 100

"Program" and "Verifikasi" also move the programming or verification date of the radio forward.

GET /radios/free-ids

Read · the next free Radio IDs

Parameters: `count` (1–50, default 5) and `after` (only IDs above this number). Free means not used by an active radio, not reserved by an archived one, and with an MDC Code without F.


```
{ "data": [ { "dmr_id": 11, "mdc_code": "B" }, { "dmr_id": 12, "mdc_code": "C" }, { "dmr_id": 13,
"mdc_code": "D" } ] }
```

GET /radios/check

Read · checks a Radio ID and SN before saving

Parameters: `dmr_id`, `serial_number`, and `radio` (the id of the radio being changed, so that its own ID and SN are not counted). The answer uses the messages of the form:

```
{
  "data": {
    "dmr_id": { "value": 751, "mdc_code": "2EF", "free": false,
                "message": "Radio ID 751 cannot be used: its MDC Code (2EF) contains the letter F. Choose
another ID." },
    "serial_number": { "value": "867PRV0201", "free": false,
                       "message": "SN 867PRV0201 is already registered on ID 201 · 867PRV0201." }
  }
}
```

GET /radios/export

Read · an Excel file (.xlsx), 10 a minute per token

Like the Export button on the page: every radio that matches the filters of `GET /radios`, without pages. The answer is an Excel file:

```
Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
File name:    register-radio-YYYYMMDD-HHMMSS.xlsx
```

GET /radios/{radio}/checkout-form

Read · the content of the IT Equipment Checkout Form as data

For applications that print the form their own way. The keys of `fields` are the boxes of the form; the signature of the ICT officer is not sent through the API.

```
{
  "data": {
    "radio_id": 6,
    "form": { "number": "MTI-HRM-ICT-FRM-001", "revision": "Rev.000", "title": "IT EQUIPMENT CHECKOUT
FORM" },
    "category": "HT", "struck": "RIG", "date": "19-10-2026",
    "fields": { "requestor_name": "Andi Saputra", "radio_id": "201", "radio_type": "MotoTrbo XiR C2660",
                "radio_serial": "867PRV0201", "user_name": "Andi Saputra", "ict_date": "19-10-2026" }
  }
}
```

7. History and import

GET `/history`

Menu Radio History · Read · the history of every radio, newest first

Parameter	Content
`search`	In the description and document no., and the SN, user and Radio ID of the radio (max. 100)
`action[]`	Kinds of history (see `history_kinds` in `/references`), several allowed
`date_from`, `date_to`	The range of event dates
`page`, `per_page`	Pages

Every line has the same form as in `GET /radios/{radio}/history`.

GET `/import`

Menu Import Excel · Read

The report of the last import (rows read, radios imported, rows skipped with their reason, data marked "Needs checking") and the number of radios in the register now. `last_import` is `null` when there has never been an import.

POST `/import`

Menu Import Excel · Create · multipart/form-data, 10 a minute per token

Field	Required	Content
`file`	Yes	The old register as .xlsx, max. 20 MB
`fresh`	–	`1`: first delete the register and its history; also needs Delete on Register Radio
`konawe`	–	`0`: skip the sheet "to Konawe" (radios handed over to Konawe, imported as archived)

An import only fills an empty register: when the register already holds radios it is refused (422), unless `fresh=1`. An import with `fresh=1` is also refused (422) while a Service & Repair ticket is open or a Stock Opname session has not been sent to the register; the message gives their number. One import at a time (409 while another runs or a radio is being saved). The answer is the report of that import.

> `fresh=1` deletes every radio and every line of history recorded in the application. Never use it on a register in use.

8. Master data

Four master tables share one pattern of endpoints. Replace `{master}` with `radio-types`, `companies`, `departments` or `channel-templates`; each table belongs to its own menu under Radio Master Data (Radio Types, Companies, Departments, Channel Templates).

Method	Endpoint	What it does
GET	`/{master}`	The list (`search`, `sort`, `direction`, `page`, `per_page`); every row carries `usage`, the number of radios (active or archived) that use it
GET	`/{master}/{id}`	One row
POST	`/{master}`	Adds a row; a 201 answer. A code or name that must be unique is refused (422) when it is taken
PUT	`/{master}/{id}`	Changes a row. `is_active: false` deactivates it: radios that use it keep it, but it can no longer be chosen
DELETE	`/{master}/{id}`	Deletes a row no radio uses; a 204 answer. While it is still used: 422, deactivate it instead

Fields per table

Table	Fields (required ones marked *)	Sorting (`sort`)
`radio-types`	`name`* (max. 100), `category`* (`HT` or `RIG`), `power` (max. 30), `is_active`	`name`, `category`, `power`
`companies`	`code`* (max. 50, stored in capitals), `name` (max. 150), `is_active`	`code`, `name`
`departments`	`name`* (max. 100, stored in capitals), `is_active`	`name`
`channel-templates`	`name`* (max. 100), `channels` (the channels, max. 2000), `effective_date` (valid from), `is_active`	`name`, `effective_date`

```
GET /api/v1/radio-types?per_page=2
{ "data": [ { "id": 3, "name": "MotoTrbo XiR C2660", "category": "HT", "power": "5 W", "is_active": true,
"usage": 14 }, ... ],
  "links": { ... }, "meta": { ... } }
```

Radios last programmed before the `effective_date` of their channel template are marked `reprogram` (needs reprogramming).

9. Users, roles and audit log

All three can only be read through the API; they are changed on their pages.

GET `/users` · `/users/{user}`

Menu Registered Users · Read · the same reach per company as on the page

List parameters: `search` (name and e-mail), `status` (`active` or `inactive`), `role` (role id), `sort` (`name`, `email`, `last_login_at`), `direction`, `page`, `per_page`.

```
{ "id": 5, "name": "Andi Pratama", "email": "andi.pratama@example.test",
  "company": { "code": "MCG", "name": "PT Merdeka Copper Gold" }, "roles": [], "active": true,
  "sign_in_method": "Pending", "directory_username": null,
  "last_sign_in_at": null, "last_sign_in_method": null, "created_at": "2026-10-17T12:33:30+08:00" }
```

`sign_in_method`: LDAP, Microsoft, Both or Pending. `last_sign_in_method`: Password, LDAP or Microsoft.

GET `/roles` · `/roles/{role}`

Menu Role & Permission · Read · the Super Admin role is only visible to Super Admin

The roles, with the number of people who hold them (API clients not counted); parameters `search`, `sort` (`id`, `name`), `direction`, `page`, `per_page`. One role holds every menu it opens, with its permissions (Read, Create, Update, Delete).

```
{ "id": 2, "name": "Admin Radio", "home_page": { "id": 1, "title": "Dashboard", "route": "dashboard.index"
},
  "scope": "ALL", "people_count": 0 }
```

GET `/audit-log`

Menu Audit Log · Read · newest first

Parameter	Content
`search`	The e-mail of who acted, the name of their account now, or what was typed on a failed sign-in
`action`	One action, for example `auth.login_failed` or `radio.updated`
`date_from`, `date_to`	A range of dates
`entity_type`, `entity_id`	The trail of one record, for example `entity_type=Radio&entity_id=12`
`page`, `per_page`	Pages

```
{ "id": 113, "at": "2026-10-19T16:30:08+08:00", "user": { "id": null, "name": "System", "email": null },
  "action": "api.token_created", "activity": "created API token",
  "entity": { "type": "ApiToken", "id": 8, "label": "HTTP check · Browser Check" },
  "old_values": null,
  "new_values": { "Name": "HTTP check", "Owner": "browser.check@example.test", "Access": "Read & write",
"Expires": "18-11-2026" },
  "ip_address": null, "via": null }
```

The recorded values are already masked: no passwords, tokens or signatures in them. `via` holds "API · <token> (#id)" for changes through the API, `null` for changes made on the pages.

10. SharePoint documents

The COF scan folder and the ICT Masterlist workbook are read with the application's Microsoft identity, as on their pages: who may see them is decided by the application's menus, not by a person's rights in SharePoint.

- SharePoint content is kept for a few minutes; `refresh=1` asks again when the copy is older than 15 seconds.
- `stale: true` means that SharePoint cannot be reached right now and the previous copy is sent.
- Errors from SharePoint carry a `reason`: `unavailable`, `forbidden`, `not_found`, `too_large`, `too_slow`, and others.
- The limit is 30 requests a minute per token.

GET `/documents/cof-scan`

Menu COF Scan · Read · the content of the scan folder, folders first, then files, newest first

Parameters: `path` (a subfolder, as the list gives it), `search`, `code` (a document number, for example `ICT/COF/123` or `ICTCOF123`), `sort` (`modified`, `name`, `label`, `extension`, `size`, `masterlist`), `direction`, `page`, `per_page`, `refresh`.

When the token may also open the COF sheet of the ICT Masterlist, every file carries `masterlist`: `linked` (a row names it), `differs` (a row has its number but another name) or `unlinked`, with the Excel row and its notes.

GET `/documents/cof-scan/files/{id}`

Menu COF Scan · Read · downloads one file

The file as it is in SharePoint: a PDF or a picture of up to 10 MB. Only files in the list of their folder (`id`, and `path` for a subfolder). Every download is written to the application log (who, which token, which file).

GET `/documents/ict-masterlist`

Menu ICT Masterlist · Read · the workbook and its sheets

Every sheet with its columns and its number of rows. `editable`: whether this token may request cell changes (Update on the ICT Masterlist menu and a read & write token). Parameter: `refresh`.

GET `/documents/ict-masterlist/rows`

Menu ICT Masterlist · Read · the rows of one sheet

Parameter	Content
`sheet`	The name of the sheet, or its position (from 0); the first sheet by default
`search`	Words to look for in every column
`status`	On the COF sheet: only rows with the status ok, `missing`, `differs` or `below`
`line`	Only these Excel rows, separated by commas (max. 20)
`sort`, `direction`, `page`, `per_page`, `refresh`	Sorting, pages, and asking again

Cells are sent as the page shows them (dates as dd-mm-yyyy, numbers with a decimal comma), with `line` as the Excel row number; by default the last row comes first. On the COF sheet the last column is "File scan", and every row carries `scan` (`ok`, `missing`, `differs` or `below`); `sheet.scan` counts them. For a token that may request changes, `locked` names the columns that hold formulas, and `waiting` the cells still waiting for a decision (column → new value).

POST `/documents/ict-masterlist/edit-requests`

Menu ICT Masterlist · Update · requests a change to one cell; a 201 answer

Field	Required	Content
`sheet`	Yes	The position of the sheet (from 0), as `sheet.index` in the rows answer
`line`	Yes	The row number in Excel
`column`	Yes	The position of the column in the table (from 0)
`value`	–	The new value as typed on the page: a number 1500 or 1.500,50, a date dd-mm-yyyy; empty to clear the cell

Nothing is written to the workbook yet: the request waits for someone else's approval. It is refused when the value equals the content of the cell, the cell holds a formula, or the cell still has a request waiting (409).

GET `/documents/edit-requests` · `/documents/edit-requests/{editRequest}`

Menu Edit Approvals · Read · cell change requests, newest first

List parameters: `status` (`pending`, `processing`, `applied`, `rejected`, `conflict` or `failed`), `search` (words in the cell, sheet, column, values, names or row key), `page`, `per_page`. `counts` counts everything that matches `search`, per status; `may_decide` says whether this token may approve and reject.

POST /documents/edit-requests/{editRequest}/approve

Menu Edit Approvals · Update · personal tokens only, 10 a minute

The workbook is read as it is now, and the cell is only written when the cell and its row still hold what they held when the change was requested, and nobody saved the file in between. The answer is the request as it ended:

Code	Result
200	`applied`: the cell is written to SharePoint
409	`conflict`: the cell or its row has changed, nothing is written; request it again. Also for a request that has already been decided
502	`failed`: SharePoint could not be written (for example the file is open in desktop Excel); it can be approved again
403	Your own request, or an API client token

POST /documents/edit-requests/{editRequest}/reject

Menu Edit Approvals · Update · personal tokens only

Rejects a request that is pending, failed or in conflict; nothing is written. Optional field `note` (max. 500): the reason shown to whoever requested the change.

11. E-Sign

The signatures of the ICT team for the PROCESSED BY box of the checkout form. Signature pictures are never sent through the API; registering, replacing and reactivating a signature are only done on the E-Sign Master page.

GET /e-signs

Menu E-Sign Master · Read

The list of signatures: their owner, the printed name, whether they are active, and how often they were used. Parameters: `search` (the printed name, and the name and e-mail of the account), `page`, `per_page`.

GET /e-signs/{signature}/uses

Menu E-Sign Master · Read

The checkout forms the signature was used on (opened by its account), newest first: the last 100, and the total.

POST /e-signs/{signature}/deactivate

Menu E-Sign Master · Update

Deactivates a signature, for example when its person leaves: forms opened by the account get an empty box again, to be signed on paper. A signature that is already inactive stays inactive.

12. Health check

For monitors (uptime monitors, load balancers, container healthchecks): whether the application works normally down to its database and, with a token, how each service it uses is doing. Neither needs a menu permission.

GET `/up`

No token · outside `/api/v1` : `https://<app-address>/up`

The application answers, without touching the database, a session or a token. Used by the healthcheck of the Docker container (`docker-compose.yml`), which restarts the container when it fails. A database that is down does not show here: use `/api/v1/health` for that.

```
{ "status": "up" }
```

GET `/health`

Without a token: the status · with any API token (read only and API clients too): details per service

Without a token, the application runs the four checks that make the website work:

Check	What is checked
`web`	The login page is opened as a browser opens it (through routing, the session, the language and the templates) and shows with its form
`database`	The database answers, and the application's tables (`users`, `menu`, `radios`) can be read; nothing is written
`storage`	The application's storage folder can be written and read (the test file is removed at once)
`session`	A session can be saved and read back where the sessions of the pages are kept

The overall `status` is `ok` , or `down` when one of the four fails. A `down` answer has the code **503**, so a monitor that only looks at the HTTP code is enough. The answer is not cached (`Cache-Control: no-store`).

```
GET /api/v1/health                                                  → 200
{
  "data": {
    "status": "ok", "version": "1.21.4", "time": "2026-10-21T23:15:19+08:00",
    "checks": { "web": { "status": "ok" }, "database": { "status": "ok" },
                "storage": { "status": "ok" }, "session": { "status": "ok" } }
  }
}
```

With a token, every check is shown in full: `status` (`ok` ; `warn` : works, but needs a look; `fail` ; `skipped` : not set up on this server), `ms` (how long the check took), and when it is not `ok` also `reason` (a fixed word, for programs) and `message` (a sentence in the language of the request). Besides the four checks above:

Check	What is checked
`cache`	The cache can be written and read (`store`: its kind)
`migrations`	Migrations not yet run on the database: `pending` (their number) and `names` (their names). Any pending: `warn`
`snipe_it`	The Snipe-IT API accepts the application's token (`version`: the version of Snipe-IT); it is only read

Check	What is checked
`microsoft`	The application can sign in to Microsoft, which is used for Microsoft sign-in and SharePoint
`ldap`	Active Directory can be asked with the application's account
`hr_database`	The HR database (looking up employees when an account is added) answers

When one of these checks is `fail` or `warn` while the four essential checks work, `status` becomes `degraded`, with the code 200: the website works, but a supporting service needs a look. An example from a copy of the application that does not use Microsoft and cannot reach its Active Directory:

```
GET /api/v1/health    (Authorization: Bearer <token>)               → 200
{
  "data": {
    "status": "degraded", "version": "1.21.4", "time": "2026-10-21T23:15:19+08:00",
    "checks": {
      "web": { "status": "ok", "ms": 8 },
      "database": { "status": "ok", "ms": 2 },
      "storage": { "status": "ok", "ms": 9 },
      "session": { "status": "ok", "ms": 11, "driver": "file" },
      "cache": { "status": "ok", "ms": 1, "store": "array" },
      "migrations": { "status": "ok", "ms": 10, "pending": 0 },
      "snipe_it": { "status": "ok", "ms": 30, "version": "v6.2.4-pre" },
      "microsoft": { "status": "skipped", "ms": null, "reason": "not_configured",
                     "message": "Not set up here." },
      "ldap": { "status": "fail", "ms": 55, "reason": "unavailable",
                "message": "Cannot be asked: not reached, or its credentials were refused." },
      "hr_database": { "status": "ok", "ms": 0 }
    }
  }
}
```

reason	Used by	message
`not_configured`	`snipe_it`, `microsoft`, `ldap`, `hr_database` (skipped)	Not set up here.
`unreachable`	`database`, `snipe_it`, `microsoft`, `hr_database`	Cannot be reached.
`unavailable`	`ldap`	Cannot be asked: not reached, or its credentials were refused.
`refused`	`snipe_it`, `microsoft`	Refused the credentials.
`failed`, `not_found`	`snipe_it`	Answered with an error. / Not found.
`not_writable`	`storage`, `session`, `cache`	Cannot be written.
`not_working`	`web`	The login page is not served as it should be.
`tables`	`database`	The tables of the application cannot be read.
`pending`	`migrations` (warn)	N migrations have not been run on this database yet.
`no_table`, `database`	`migrations`	The table of the migrations is missing. / Not checked: the database does not answer.

> The answers of the health check never name a server, an address or an error message of a server: the cause is written to the application log. A wrong, revoked or expired token gets 401. When the database is down, so that the token cannot be checked, the answer still comes, without the details.

13. Example workflows

Monitoring the application

- Call `GET /api/v1/health` without a token every 1–5 minutes, and raise an alert when the code is not 200.
- To find the cause, call it again with a read only token, for example the token of a monitoring API client: the checks that `fail` give their `reason`. A `degraded` status means that the website works, but a supporting service (for example Active Directory or Snipe-IT) has a problem, or a migration has not been run yet.
- A container healthcheck only needs `/up`.

Registering a new radio

1. Get a free Radio ID: `GET /radios/free-ids?count=1`.
2. Check its SN: `GET /radios/check?dmr_id=11&serial_number=867ABC0001`.
3. Register it: `POST /radios` with `dmr_id`, `serial_number`, `radio_type_id`, `placement` and `condition`. The 201 answer holds the new radio with its `id` and `version`.

Handing a radio over to a user

1. Read the radio: `GET /radios/{id}`, and keep its `version`.
2. Change it: `PUT /radios/{id}` with `user_name`, `company_id`, `department_id`, `placement: "di_user"`, `condition`, `document_no` and `version`. A "Serah Terima" (handover) line of history is recorded automatically.
3. When the answer is 409, read the radio again and repeat from the latest data.

```
PUT /api/v1/radios/6
{ "user_name": "Andi Saputra", "company_id": 3, "department_id": 7, "placement": "di_user",
  "condition": "layak", "document_no": "BA2300012", "version": "20261018235242190" }
```

Recording a verification (stock taking)

For every radio checked: `POST /radios/{id}/history` with `action: "Verifikasi"`, `event_date` and `description`. The verification date of the radio moves forward too. Radios that were never verified can be found with `GET /radios?issue=verify_empty`.

Fetching the whole register for a report

- One file: `GET /radios/export` with the filters needed (10 a minute).
- As JSON: `GET /radios?per_page=100&page=1`, then follow `links.next` until it is `null`. Add `scope=all` to include archived radios.
- For Power BI, create an **API client** with a read only role and a read only token, then call the endpoints above with the `Authorization` header.

> Endpoints and fields not described in this document are in the interactive documentation `https://<app-address>/docs/api`, which always matches the version of the running application.